

Precon: A Precision-Convertible Architecture for Accelerating Quantized Deep Learning Models across Various Domains Including LLMs

Jongwoo Park*, Hyeonseong Kim*, Jiyun Han, and Seungkyu Choi†

Kyung Hee University, Republic of Korea

{pmk0638, hsyoun2222, hjy0013, seungkc}@khu.ac.kr

Abstract—The sensitivity of LLMs to quantization has driven the development of hardware accelerators tailored for specific low-precision configurations such as weight-only quantization and mixed-precision, which can introduce inefficiencies in dedicated hardware architecture. In this work, we propose *Precon*, a precision-convertible architecture designed to accelerate various quantized deep learning models, particularly LLMs, through a unified processing unit. By enabling on-the-fly switching between half-float (FP16) decoding and integer (INT) decomposition, the design effectively supports INT4-FP16, INT4-INT4, and INT4-INT8 arithmetic within shared logic. *Precon* achieves up to $4.1\times$ speedup and 81.4% reduction in energy consumption compared to the baseline across various domains, including the support of both accurate and efficient acceleration of quantized LLMs.

Index Terms—large language models, quantization, systolic array, floating-point arithmetic, mixed-precision.

I. INTRODUCTION

Deep learning models have shown exceptional performance across various AI domains, including computer vision [1], natural language processing (NLP) [2], robotics [3], and autonomous systems [4]. In particular, recent years have seen large language models (LLMs) achieve state-of-the-art performance in many NLP tasks. LLMs have become deeply integrated into daily life, accessible through applications such as text generation, chatbots, and machine translation [5]–[7]. However, the vast number of parameters in LLMs results in significant memory and computational demands, which limits their broader applicability [8].

One of the promising solutions to alleviate the addressed burden is data quantization [9], [10]. While deep learning models in various AI domains such as computer vision (e.g. image classification, object detection, super-resolution, etc.) or simple language processing tasks (e.g. task classification, question answering, multi-modal, etc.) can adopt ultra-low-precision computing using sub-8-bit integer (INT) formats [11]–[13], typical language processing tasks based on LLMs suffer from accuracy degradation in low-precision computing. Consequently, various research targeting post-training quantization (PTQ) of LLMs have been actively studied to support accurate low-precision LLMs.

Recent studies have shown that quantized LLMs face accuracy challenges due to the broad data distribution observed in

activation tensors [14]. Consequently, weight-only quantization approaches, such as applying aggressive INT4 quantization to weights while keeping activations in half-precision (FP16), have become one of the mainstreams in LLM quantization [15]–[18]. Recent research also explores INT-only quantization methods employing mixed-precision techniques, such as 4-bit weights and 8-bit activations [19], [20], or aggressive full 4-bit quantization techniques [19], [21]. However, these methods require intensive calibration processes to reshape data distributions and often result in noticeable accuracy degradation, posing challenges for widespread adoption in practical applications.

Given this context, specialized processing unit designs have become essential on the hardware side to accelerate inference using quantized LLMs. A recent approach proposes an INT-FP arithmetic-based accelerator designed to support weight-only quantized LLM models [22]. It substitutes FP arithmetic with INT arithmetic in the processing array, allowing approximation through mantissa truncation within a shared exponent group. However, this approach lacks support for lower-precision INT-only arithmetic, restricting its applicability to a broader range of quantized models beyond weight-only quantization. On the other hand, there are several designs of precision-scalable processing units that are available to support mixed- or low-precision INT-based quantized models [23], [24]. While effective for INT-based computations, these designs lack support for high-precision FP arithmetic, limiting their utility for high-accuracy LLMs. Thus, flexible support of both INT and FP arithmetic is essential to fully accommodate quantized deep learning models, including reliable LLM serving.

In this work, we introduce *Precon*, a precision-convertible processing unit architecture designed to accelerate various quantized deep learning models, including LLMs. To support floating-point-based arithmetic required for reliable LLM inference, *Precon* incorporates on-the-fly FP16 to INT conversion within the processing unit, enabling both INT-INT and INT-FP arithmetic across multiple precision levels. This unified processing architecture supports low-precision inference with INT4-INT4/8 for error-resilient models—including LLMs, convolutional neural networks (CNNs), and transformer encoders—as well as high-precision inference with INT4-FP16 for high-accuracy LLMs. We evaluate the hardware efficiency of our systolic array architecture on various quantized models,

* J. Park and H. Kim contributed equally to this work.

† S. Choi is the corresponding author.

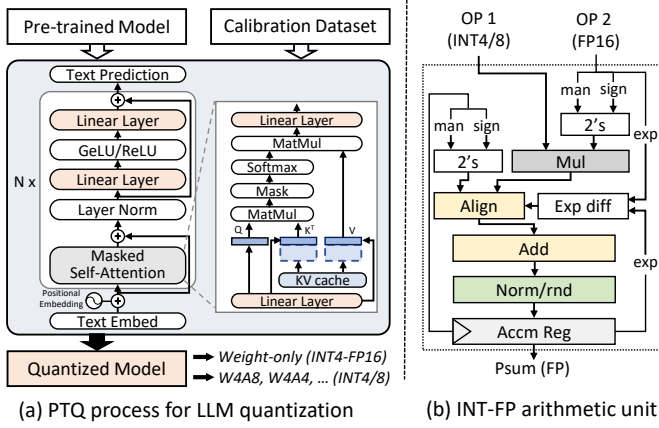


Fig. 1. (a) An overview of PTQ process for LLMs, (b) an INT-FP arithmetic unit for weight-only quantized LLMs.

demonstrating its effectiveness by comparing it to both the FP16 baseline and the state-of-the-art INT-FP accelerator [22].

Our contributions are summarized as follows.

- A decomposable FP16 encoding method is proposed to effectively decode the data into four INT4 bricks, enabling separate multiplication and addition with minimal logic overhead from precision conversion.
- We design a unified processing unit maximally leveraging the same logic components for computing different precision levels.
- We analyze the area and power efficiency of the processing unit design, followed by an evaluation of the speedup and energy efficiency of the systolic array, demonstrating improved hardware efficiency and versatility.

II. BACKGROUND

A. Low-Precision Computing for LLMs

With constraints on training data and high computational demands, quantizing LLMs often relies on PTQ. Fig. 1(a) describes the overall PTQ process to obtain low-precision LLM serving. Given the extensive number of weight parameters, which dominates the memory usage, and the challenges in reducing activation bit-width posed by outliers, weight-only quantization is a commonly adopted approach in LLM quantization [15]–[18]. This approach quantizes only the weights to extremely low precision, such as INT4, while preserving high-precision computation with the activation data. It offers reduced memory consumption while maintaining near full-precision accuracy across various LLM tasks.

Recent research has shown that optimization techniques addressing outliers during calibration can enable low-precision INT arithmetic for LLM inference with acceptable accuracy. Various methods such as data redistribution through channel assembly [14], [19] and rotation-based redistribution [21], [25], have been proposed to mitigate the outliers observed in activation channels. These state-of-the-art techniques demonstrate that mixed-precision configuration of INT4 for weights and INT8 for activations yields reliable LLM performance with minimal accuracy drop. Furthermore, aggressive INT4-only

quantization has been explored, accepting a certain accuracy drop. Despite the accuracy degradation, these recent attempts have triggered the possibility of INT-only arithmetic for LLM inference.

B. INT-FP Arithmetic for Quantized LLMs

LLM inference requires processing through a sequence of Transformer decoder blocks, with linear layers comprising the majority of the computational workload [22]. To optimize this workload, dedicated arithmetic units have been developed to accelerate the matrix multiplications in these linear layers, tailored to the precision configuration of the operands, namely weights and activations. While INT-only configurations can be efficiently managed with straightforward INT arithmetic, weight-only quantization introduces complexity by necessitating specialized arithmetic that handles INT-FP16 multiplication for the mixed-precision operands and FP-based accumulation.

The conventional approach to supporting weight-only quantization utilizes FP16 arithmetic with on-the-fly format conversion from INT to FP16. While this method optimizes memory usage, it fails to address computational inefficiencies. To fully leverage the advantages of quantized weights, a prototype of multiply-and-accumulate (MAC) unit specially designed for INT4-FP16 operations can be employed, as illustrated in Fig. 1(b). This unit directly accepts the INT operand and multiplies it with the mantissa of the FP16 operand, thereby significantly simplifying the FP-based multiplier. FP-based addition is then used to accumulate partial sums in FP16 format.

While such design can significantly reduce area and power consumption compared to the FP16-based design, it has a critical limitation in that it cannot fully leverage hardware efficiency for INT-INT low-precision computing. Given the need for INT-FP arithmetic to maintain accuracy in LLM inference, alongside INT-INT arithmetic support for models that allow further quantization, a reconfigurable MAC unit architecture is essential. This would enable effective handling of low-precision configurations commonly employed across diverse AI domains.

III. PRECON ARCHITECTURE

A. Convertible FP16 Data Format

The key requirement for supporting both INT and FP formats in the same processing engine is the ability to decompose an FP representation into uniform INT data bricks. If FP16 format is segmented into four 4-bit bricks, each capable of participating in INT-based arithmetic, efficient hardware-level switching between INT and FP arithmetic modes becomes feasible. Accordingly, we slightly modify the standard FP16 format, organizing it into one 4-bit brick for the exponent and three 4-bit mantissa bricks, including the sign bit.

Referring to the Bit Fusion technique [23], the decomposed bricks in a single data represent different bit positions, requiring the need for 4-bit shifts to align position for proper arithmetic operations. Similarly, FP addition involves a shift operation for alignment based on the exponent difference between operands. To streamline these processes, we encode the exponent as a

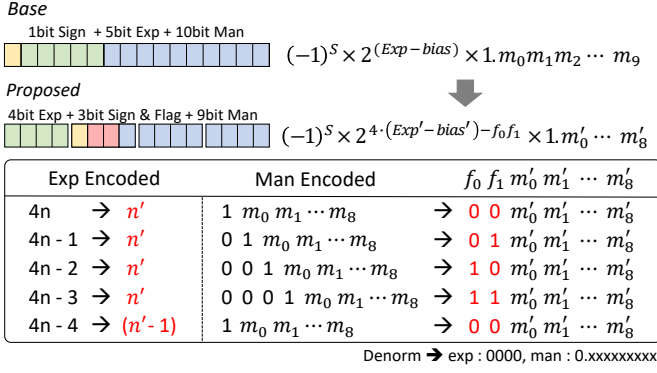


Fig. 2. The decomposable fourfold exponent-encoded FP16 format.

multiple of four, enabling the use of 4-bit shifters to perform alignment in both FP addition and bit-brick fusion efficiently.

Fig. 2 illustrates the proposed FP16 format, which employs a fourfold encoding for the exponent. Although this approach reduces the exponent field to 4 bits, it effectively offers an expanded range nearly twice as wide as the standard FP16's 5-bit exponent, demonstrating feasibility for a broader range of values. As each consecutive exponent value differs by a factor of four, the mantissa must represent intermediate values within the normalized range. Therefore, a leading-one must be embedded within the mantissa bits to represent the full normalized range. As shown in the left table of Fig. 2, the position of the leading-one varies based on the remainder of the exponent divided by four. This results in four possible cases, corresponding to remainders of 0 through 3, each of which is encoded using a 2-bit flag to compactly represent the leading-one position.

While a similar approach of FP16 decomposition has been studied [26], it fails to capture the leading-one value effectively inside the mantissa representation leading to a significant error produced by unavoidable mantissa truncation. On the other hand, the proposed 2-bit flag encoded inside the mantissa can effectively represent by only giving up a single bit in mantissa from the standard FP16 in every representation. Besides, the wider exponent range can even enhance the accuracy of LLM inference with possessing excessive data range [27].

B. Processing Unit Design

Given the proposed format as one of the MAC operands, we introduce a format-convertible processing unit that executes both INT4-FP16 and INT4-INT4 $\times$ 4 MAC operations by maximizing the shared use of computing logic. Fig. 3 presents a simplified block diagram of the MAC unit. The unit comprises four INT4 multipliers, an aligner-combined adder module, four distinct position registers, and an aggregator unit that collects data from each register to reformulate the output appropriately.

Multiplication After the data decomposition, all the input operands are represented as 4-bit bricks to engage in INT-based multiplication. Due to the encoded mantissa using the flag bits, a format decoder is required to produce three bricks, including those representing leading-one. In INT-INT mode, four separate multiplied data bricks are generated and accumulated in each

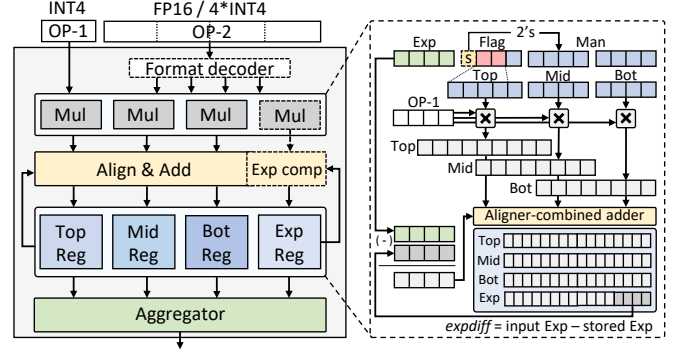


Fig. 3. The processing unit architecture with operation details.

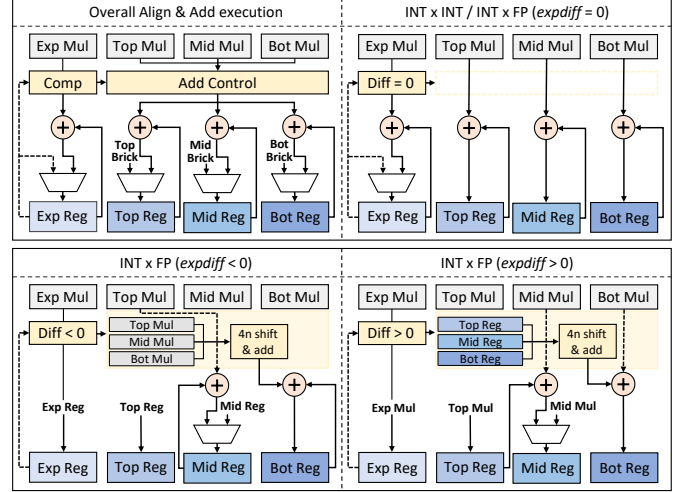


Fig. 4. The logical datapath of the proposed aligner-combined addition.

register. In INT-FP mode, three multipliers handle the mantissa portion, producing three multiplied data bricks, while the other single brick conveys exponent data to its designated register.

Aligner-Combined Addition Precise control logic is implemented to add the components and store them in designated position registers. These registers, labeled as exponent, top, middle, and bottom, serve dual roles: acting as standard, independent accumulators in INT mode and as designated position accumulators required for FP16 reconstruction in the final step. As shown in the middle part of Fig. 3, the aligner needed for FP16 addition is integrated into the adder unit, facilitating both INT-based and FP16-based accumulation by aligning the candidates for addition to their appropriate positions. Specifically, all the data bricks including the multiplied results and the position registers participate as potential addends and are aligned according to the calculated exponent difference ($expdiff$), which serves as the control signal, ensuring accurate placement for addition.

Fig. 4 details the logical path of the proposed accumulation method, in which four adders perform up to four separate accumulations. The adder control unit supports flexible operand positioning, allowing multiplied results and register-stored values to align with their designated positions. In INT-INT mode, addition proceeds by directly accumulating multiplied results into the corresponding registers. For INT-FP mode, the

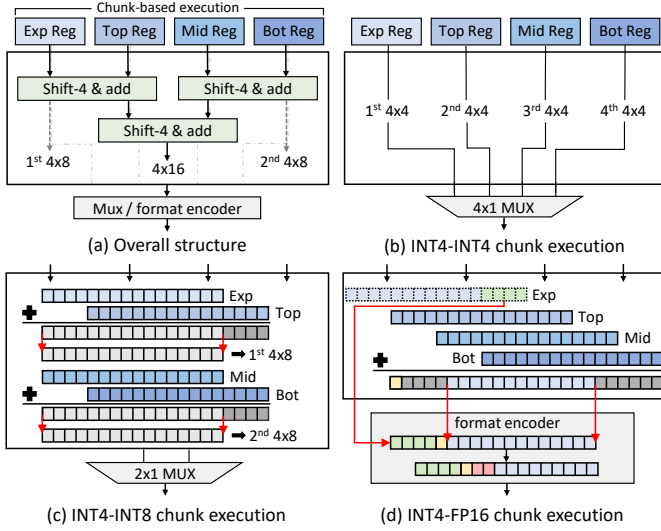


Fig. 5. Operation details of the precision-convertible aggregator.

operation branches into three cases based on the *expdiff* value between adjacent operands to be added.

When *expdiff* is zero, the operation mimics INT-INT mode but with the exponent register simply retaining the same value. For *expdiff* < 0, it indicates that the register-stored values act as base values while incoming multiplied data must be aligned. Therefore, the multiplied results serve as addend bricks and are added into registers at a position lower by *expdiff* levels. Bricks unable to lower further are shifted by 4 bits and added to the lowest, which is the bottom register. Conversely, for *expdiff* > 0, where the incoming data have a larger exponent, the multiplied results become the base values, and stored register values act as addends. The rest of the adding process follows the same rule as in the *expdiff* < 0 case. Due to the quite congested logic within the addend controls, the critical path of our design lies here; however, it remains considerably shorter than that of unpipelined general FP16 addition, as will be discussed in Section IV-C.

Aggregation The position registers can accumulate values independently until the output is requested. At that point, the aggregator unit collects the stored values across all registers to formulate the output based on the operating mode. The detailed aggregation process is shown in Fig. 5, illustrating the overall structure, which includes three 4-bit shift-and-add units to manage precision reformatting. The aggregator is triggered once per accumulation chunk, where the chunk size can vary—from a single data point to a larger grouping, such as a quantization group or a size specific to the dataflow requirements.

Fig. 5 (b)-(d) outlines the aggregation process for each mode. For INT4-INT4, the aggregator functions as a selector, simply forwarding the chosen output. For INT4-INT8, each of the 1st-stage shift-and-add operators produces an accumulated 4×8 output and sends it to the selector. Lastly, in INT4-FP16 mode, the aggregator performs a dedicated normalization process. A two-stage shift-and-add operation combines all position register values into a single representation and trims the mantissa bits as needed, outputting the final 16-bit FP format.

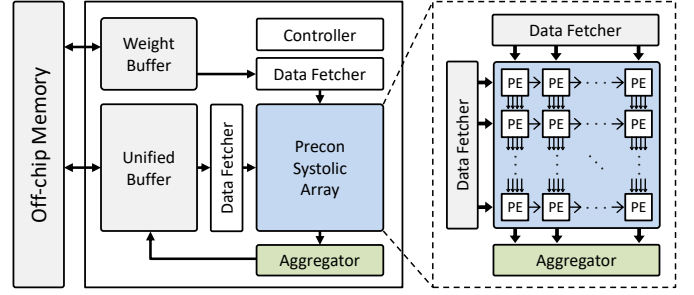


Fig. 6. High-level overview of *Precon* architecture.

C. Overall Architecture

Fig. 6 presents the overall architecture of *Precon*. A 2D systolic array is implemented with output stationary dataflow, utilizing the proposed MAC units. Data fetchers are implemented between the buffer and array to feed input activations and weights in proper timing. The unified buffer stores both input and output activation data and streams them into the array, while the weight buffer stores quantized weights. Unlike the structure of a single MAC unit, the aggregator is positioned only at the end of each column in the array, where stored data in the registers are transferred down the column to the aggregator unit, which then finalizes the output. *Precon* offers three precision modes—INT4-FP16, INT4-INT4, and INT4-INT8—enabling flexible format conversion to meet the precision requirements of various quantized deep learning models.

IV. EVALUATION

A. Experimental Setup

Quantized Models We adopt state-of-the-art PTQ frameworks of weight-only quantization [15] and INT4/8 quantization [19]. To ensure a fair comparison of accuracy across different precision levels, we use the same baseline model and reproduce PTQ models using a consistent small calibration set from the Pile dataset [28]. For the analysis of both accuracy and hardware performance, we use quantized LLM models from the OPT family [7], as well as LLaMA1-7B and LLaMA2-7B models [6]. Zero-shot accuracy of the LLMs is evaluated on six benchmarks: PIQA, BoolQ, Arc-Easy, Arc-Challenge, HellaSwag, and WinoGrande. Additionally, we evaluate and compare perplexity on WikiText-2 [29] to further assess LLM stability at different precision levels.

To broaden the evaluation across various AI domains, we also analyze the hardware performance of deep learning models under their respective state-of-the-art PTQ settings, including CNNs [30]–[32], Transformer-encoder models such as BERT [33], and Vision Transformers (ViTs) [34], [35].

Hardware Configuration A Verilog RTL model of a 32×32 systolic array of *Precon* is modeled and synthesized using Synopsys Design Compiler with a 65nm library. Power consumption is analyzed with Primetime PX (PTPX) based on post-synthesis simulations with real test vectors. On-chip scratchpad buffers are modeled using CACTI [36], and off-chip memory access is considered based on [37]. To evaluate latency

TABLE I
LLM ACCURACY ANALYSIS

	OPT 125M	OPT 1.3B	OPT 2.7B	OPT 6.7B	OPT 13B	LLaMA1 7B	LLaMA2 7B
	Average zero-shot accuracy on six benchmarks* $\uparrow$						
Baseline	43.41	51.83	54.76	59.19	59.87	63.43	63.79
INT4-FP16	40.96	50.38	55.46	59.05	60.75	63.02	64.05
INT4-INT8	42.64	51.00	54.68	56.41	47.95	62.52	64.06
INT4-INT4	36.47	35.24	35.94	34.91	35.61	48.61	48.43
	WikiText-2 perplexity (PPL) $\downarrow$						
Baseline	27.76	14.63	12.50	10.86	10.12	5.69	5.49
INT4-FP16	29.22	14.87	12.65	10.83	10.15	5.81	5.61
INT4-INT8	30.34	15.66	13.49	12.48	14.02	5.90	5.83
INT4-INT4	-	-	-	-	-	13.76	-

* PIQA, BoolQ, Arc-Easy, Arc-Challenge, HellaSwag, and WinoGrande

and memory access counts, a cycle-accurate simulator with weight- and output-stationary dataflow is employed. We first conduct a detailed analysis at the single processing unit level, followed by a comparison of our accelerator with the baseline FP16 architecture and the state-of-the-art accelerator featuring weight-only quantization [22].

B. Accuracy Analysis

We evaluate the accuracy and perplexity of various LLMs tested under different precision levels to assess their applicability. Considering the slight precision variations in FP16, which offers a wider exponent range but incurs a 1-bit mantissa truncation, these effects are incorporated into our weight-only quantized LLM PTQ simulation. Additionally, it is important to note that our processing unit module was tested on 10,000 actual test vectors, and an accumulation error of only 0.078% was observed during 128 accumulations per output using the proposed accumulation method. To account for these minor errors, random errors are injected based on a quantization group size of 128. By reproducing the accuracy of INT-based models under the same PTQ settings of INT4-FP16, we ensure a fair comparison of the reliability of various quantized LLMs.

Table I presents the LLM accuracy results using different precision levels for linear layers. As shown, the INT4-FP16 configuration demonstrates accuracy levels comparable to the baseline across a range of LLM sizes. While the zero-shot accuracy for the smallest OPT model exhibits noticeable degradation, INT4-FP16 consistently achieves the smallest accuracy drop and the lowest perplexity across all models. Furthermore, the stability of INT4-FP16 improves with increasing model size, categorizing this weight-only quantized LLMs as *accurate LLMs*. In the INT4-INT8 mode, which uses 4-bit weights and 8-bit activations, reasonable accuracy is preserved in most models. Several models even outperform INT4-FP16 in accuracy, but generally incurring slightly higher perplexity than INT4-FP16. With the integration of the data redistribution technique from [19], INT4-INT8 achieves sufficient accuracy for quantized LLMs, classifying it as *efficient LLMs*.

For INT4-INT4, the accuracy results are suboptimal in most LLMs, with accuracy drops exceeding 10% and unstable perplexity performance. Despite these poor outcomes, we do not apply additional techniques to improve accuracy, ensuring a fair comparison across all configurations. Note that recent studies

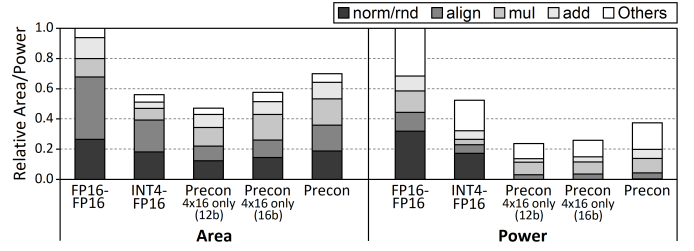


Fig. 7. Area and power comparison of a single MAC unit with breakdowns.

have highlighted the potential of rotation-based PTQ methods to enhance the performance of INT4-only models [21], [25]. Therefore, we include INT4-INT4 in our hardware evaluation for quantized LLM inference, considering as a prospective area for future adoption, contingent on further optimizations.

C. Processing Unit Efficiency

We first evaluate the performance of a single MAC unit to demonstrate the hardware efficiency of *Precon*. Comparison models of FP16 and INT4-FP16 baseline units are configured to match the same accumulation error as *Precon* by adjusting the adder bit-width, ensuring a fair evaluation. A higher clock frequency of 500 MHz is set for the single unit evaluation to highlight the efficacy of our design in terms of critical path delay compared to traditional FP arithmetic units. Additionally, we evaluate different versions of *Precon*, including an INT4-FP16 fixed mode (denoted as “4×16 only”) and a configuration with 12-bit registers instead of the default 16-bit registers.

Fig. 7 depicts the relative area and power consumption of different MAC units, including a breakdown of individual modules. FP baseline units exhibit high area occupation in the aligner and normalizer modules. Due to reduced multiplication complexity, the INT4-FP16 baseline achieves significant area and power savings compared to the FP16 baseline. *Precon*, leveraging brick-level MAC operations, replaces high-cost barrel shifters in the aligner and normalizer with fixed 4-bit shift-and-add circuits and position controls. This substitution allows additional area reductions, with the 4×16 only mode achieving a 52.9% area reduction. To accommodate separate INT accumulations and cover all precision levels supported by *Precon*, 16-bit position registers are used by default. Although this design requires additional logic for precision conversion, *Precon* remains 30.1% smaller than the FP16 baseline.

Power consumption breakdowns reveal that a significant portion of power in FP arithmetic units is consumed by the normalizer. The aggregator in *Precon*, which manages normalization and precision conversion, operates on a chunk-based approach, requiring only a single operation per chunk size. This approach enables substantial power savings, as the aggregator remains inactive during computation, consuming only a few microwatts. Consequently, *Precon* achieves power reductions of 62.6% and 28.6% compared to the FP16 baseline and INT4-FP16 baseline units, respectively.

D. Speedup and Energy Efficiency

To evaluate the overall performance and energy efficiency of deep learning models, a 32×32 systolic array of *Precon* is

TABLE II
PRECON IMPLEMENTATION SUMMARY

	Area (mm^2)	Power (W) - 4×16	Power (W) - 4×8
32×32 Array	4.024	0.849	0.750
Aggregators	0.038	0.001	0.002
Data Fetchers	0.268	0.101	0.115

	Accurate LLMs	Efficient LLMs	ResNet-18/50	Mobile-V2	Efficient-B0	BERT Models	ViT Models
Precision Level	INT4-FP16	INT4-INT8	INT4-INT4	INT4-INT8	INT4-INT8	INT4-INT4	INT4-INT8

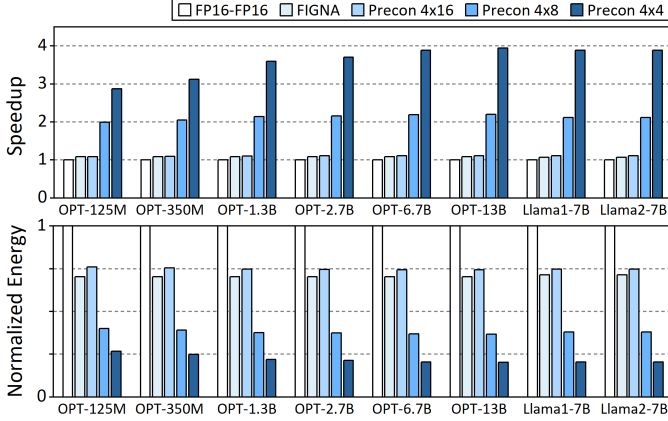


Fig. 8. Analysis of speedup and energy consumption in various LLMs.

implemented. Detailed area and power consumption are summarized in Table II, which shows slightly reduced power usage in INT modes. Two comparison architectures are prepared: the FP16-baseline and FIGNA [22], which represents state-of-the-art INT4-FP16 acceleration by using INT-only MAC units with an approximated block-based FP pre-alignment process.

Fig. 8 illustrates the speedup and normalized energy consumption of various quantized LLMs. All precision levels supported by *Precon* are evaluated, irrespective of accuracy, as discussed in Section IV-B. The throughput is consistent across the comparison models, but FIGNA and *Precon* 4×16 exhibit slightly lower latency compared to the FP16 baseline due to increased loading latency observed in the latter. As the model size increases, *Precon* achieves up to $2.2 \times$ and $3.9 \times$ speedup compared to the baseline in 4×8 and 4×4 modes, respectively. In smaller models, the limited size of linear-layer channels prevents full exploitation of parallel processing of up to four tokens in low-precision modes, resulting in lower speedup. Energy consumption follows a similar trend, with additional reductions attributed to decreased data access and chip power usage compared to the FP16 baseline. *Precon*'s INT4-FP16 mode exhibits slightly higher energy consumption than FIGNA due to higher chip power demands. While FIGNA benefits from fully INT-based MAC arrays, *Precon* provides the flexibility to support single-MAC FP arithmetic, enabling potential improvements in scalability for implementation.

To demonstrate *Precon*'s versatility, various deep learning models across AI domains are evaluated using their optimal precision levels. For models in other domains like CNNs and Transformer encoders, which are more error-resilient than LLMs, aggressively quantized inference can be considered. Based on prior PTQ studies, the lowest precision ensuring

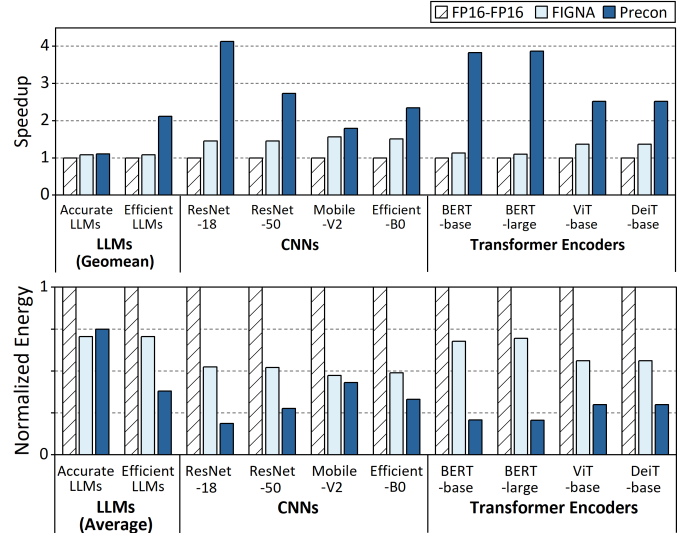


Fig. 9. Speedup and energy savings in various AI domains.

reliable accuracy is chosen for each model, as outlined in Table II. LLM results are categorized into previously defined *accurate* and *efficient* LLMs, with average values reported. Fig. 9 presents the overall speedup and energy efficiency across these models. While similar performance to FIGNA is observed in INT4-FP16 mode, *Precon* outperforms in lower precision modes, such as INT4-INT8 and INT4-INT4. This results in up to $4.1 \times$ speedup and 81.4% reduction in energy consumption compared to the baseline. These findings highlight the capability of *Precon* to adapt to diverse quantized inference requirements, spanning error-resilient CNNs to error-sensitive LLMs, through effective precision conversion in both bit-width and data format.

V. CONCLUSION

This paper presents *Precon*, a precision-convertible processing unit architecture designed to support quantized inference across a spectrum of models, ranging from error-resilient to error-sensitive, through mode conversions in both bit-width and data format. By leveraging the exponent-encoded FP16 format, which can be decomposed into 4-bit INT data bricks, the proposed MAC unit integrates an aligner-combined adder and aggregator to enable INT4-FP16, INT4-INT4, and INT4-INT8 arithmetic. *Precon* demonstrates superior hardware performance compared to existing processing unit designs, achieving up to $4.1 \times$ speedup and 82.8% reduction in energy consumption across diverse AI domains compared to the baseline systolic array architecture.

ACKNOWLEDGMENT

This work was supported in part by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (2024-00352468); and in part by Korea Collaborative and High-Tech Initiative for Prospective Semiconductor Research (K-CHIPS) funded by the Ministry of Trade, Industry and Energy (MOTIE), South Korea, under Grant RS-2024-00405946. The EDA tool was supported by the IC Design Education Center (IDEC), Korea.

REFERENCES

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Proceedings of the 25th International Conference on Neural Information Processing Systems - Volume 1*, ser. NIPS'12. USA: Curran Associates Inc., 2012, pp. 1097–1105. [Online]. Available: <http://dl.acm.org/citation.cfm?id=2999134.2999257>
- [2] A. Vaswani, "Attention is all you need," *Advances in Neural Information Processing Systems*, 2017.
- [3] C. Tang, B. Abbate-matteo, J. Hu, R. Chandra, R. Martín-Martín, and P. Stone, "Deep reinforcement learning for robotics: A survey of real-world successes," *arXiv preprint arXiv:2408.03539*, 2024.
- [4] M. R. Bachute and J. M. Subhedar, "Autonomous driving architectures: insights of machine learning and deep learning algorithms," *Machine Learning with Applications*, vol. 6, p. 100164, 2021.
- [5] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [6] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.
- [7] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin *et al.*, "Opt: Open pre-trained transformer language models," *arXiv preprint arXiv:2205.01068*, 2022.
- [8] C.-J. Wu, R. Raghuveendra, U. Gupta, B. Acun, N. Ardalani, K. Maeng, G. Chang, F. Aga, J. Huang, C. Bai *et al.*, "Sustainable ai: Environmental implications, challenges and opportunities," *Proceedings of Machine Learning and Systems*, vol. 4, pp. 795–813, 2022.
- [9] S. Hashemi, N. Anthony, H. Tann, R. I. Bahar, and S. Reda, "Understanding the impact of precision quantization on the accuracy and energy of neural networks," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2017. IEEE, 2017, pp. 1474–1479.
- [10] S. Jain, S. Venkataramani, V. Srinivasan, J. Choi, P. Chuang, and L. Chang, "Compensated-dnn: Energy efficient low-precision deep neural networks by compensating quantization errors," in *Proceedings of the 55th annual design automation conference*, 2018, pp. 1–6.
- [11] R. Banner, Y. Nahshan, and D. Soudry, "Post training 4-bit quantization of convolutional networks for rapid-deployment," *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [12] H. Tang, X. Zhang, K. Liu, J. Zhu, and Z. Kang, "Mkq-bert: Quantized bert with 4-bits weights and activations," *arXiv preprint arXiv:2203.13483*, 2022.
- [13] Z. Yuan, C. Xue, Y. Chen, Q. Wu, and G. Sun, "Ptq4vit: Post-training quantization for vision transformers with twin uniform quantization," in *European conference on computer vision*. Springer, 2022, pp. 191–207.
- [14] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "Smoothquant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning*. PMLR, 2023, pp. 38 087–38 099.
- [15] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for on-device llm compression and acceleration," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 87–100, 2024.
- [16] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "Optq: Accurate quantization for generative pre-trained transformers," in *The Eleventh International Conference on Learning Representations*, 2022.
- [17] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [18] G. Park, B. Park, M. Kim, S. Lee, J. Kim, B. Kwon, S. J. Kwon, B. Kim, Y. Lee, and D. Lee, "Lut-gemm: Quantized matrix multiplication based on luts for efficient inference in large-scale generative language models," *arXiv preprint arXiv:2206.09557*, 2022.
- [19] J. Liu, R. Gong, X. Wei, Z. Dong, J. Cai, and B. Zhuang, "Qllm: Accurate and efficient low-bitwidth quantization for large language models," *arXiv preprint arXiv:2310.08041*, 2023.
- [20] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han, "Qserve: W4a8kv4 quantization and system co-design for efficient llm serving," *arXiv preprint arXiv:2405.04532*, 2024.
- [21] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefler, and J. Hensman, "Quarot: Outlier-free 4-bit inference in rotated llms," *arXiv preprint arXiv:2404.00456*, 2024.
- [22] J. Jang, Y. Kim, J. Lee, and J.-J. Kim, "Figma: Integer unit-based accelerator design for fp-int gemm preserving numerical accuracy," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 760–773.
- [23] H. Sharma, J. Park, N. Suda, L. Lai, B. Chau, J. K. Kim, V. Chandra, and H. Esmailzadeh, "Bit fusion: Bit-level dynamically composable architecture for accelerating deep neural network," in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2018, pp. 764–775.
- [24] D. Im, G. Park, Z. Li, J. Ryu, and H.-J. Yoo, "Sibia: Signed bit-slice architecture for dense dnn acceleration with slice-level sparsity exploitation," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 69–80.
- [25] Z. Liu, C. Zhao, I. Fedorov, B. Soran, D. Choudhary, R. Krishnamoorthi, V. Chandra, Y. Tian, and T. Blankevoort, "Spinquant-llm quantization with learned rotations," *arXiv preprint arXiv:2405.16406*, 2024.
- [26] S. Choi, J. Shin, and L.-S. Kim, "A deep neural network training architecture with inference-aware heterogeneous data-type," *IEEE Transactions on Computers*, vol. 71, no. 5, pp. 1216–1229, 2021.
- [27] D. Kalamkar, D. Mudigere, N. Mellempudi, D. Das, K. Banerjee, S. Avancha, D. T. Vooturi, N. Jammalamadaka, J. Huang, H. Yuen *et al.*, "A study of bfloat16 for deep learning training," *arXiv preprint arXiv:1905.12322*, 2019.
- [28] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima *et al.*, "The pile: An 800gb dataset of diverse text for language modeling," *arXiv preprint arXiv:2101.00027*, 2020.
- [29] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," *arXiv preprint arXiv:1609.07843*, 2016.
- [30] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *CoRR*, vol. abs/1512.03385, 2015. [Online]. Available: <http://arxiv.org/abs/1512.03385>
- [31] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," 2019.
- [32] M. Tan and Q. V. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," 2020.
- [33] J. Devlin, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [34] A. Dosovitskiy, "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929*, 2020.
- [35] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," in *International conference on machine learning*. PMLR, 2021, pp. 10 347–10 357.
- [36] N. Muralimanohar, R. Balasubramanian, and N. P. Jouppi, "Cacti 6.0: A tool to model large caches," *HP laboratories*, vol. 27, p. 28, 2009.
- [37] N. P. Jouppi, D. H. Yoon, M. Ashcraft, M. Gottscho, T. B. Jablin, G. Kurian, J. Laudon, S. Li, P. Ma, X. Ma *et al.*, "Ten lessons from three generations shaped google's tpuv4i: Industrial product," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 1–14.